

Object Oriented Programming - CBP

PROPERTY RECOMMENDATION SYSTEM

Presented by

Vaishnavi Cherukuri (24071A3259)

Vavilla Dharani (24071A3260)

Veda Nampally (24071A3261)

Veerabhadra Yerram (24071A3262)

Rudra Gayatri (25075A3206)

Object Oriented Programming - CBP

GUIDING FACULTY

Mr. Indurthi Ravindra Kumar

Assistant Professor
Dept. of CSE & CSBS

Mrs. Alli Madhavi

Assistant Professor
Dept. of CSE & CSBS

Abstract

The Property Recommendation System (PRS) is **designed to streamline the process of discovering and managing real estate properties** through intelligent automation. In today's competitive property market, users often face challenges in identifying options that align with their specific preferences and financial limits. To address this, PRS offers a smart, data-driven platform that suggests the most relevant properties based on parameters such as location, budget, and property features.

Developed using **Object-Oriented Programming (OOP) concepts**—such as inheritance, polymorphism, and encapsulation—the system ensures modularity, scalability, and easy maintenance. A MySQL database efficiently manages all user, property, and recommendation-related information while maintaining data integrity and security.

By integrating automation with personalized search capabilities, the Property Recommendation System enhances convenience, accuracy, and transparency—providing a modern digital solution for effective and user-friendly real estate management.

Introduction

The growing demand for real estate has made property search and management more challenging than ever. Prospective buyers and tenants often struggle to find reliable and well-matched properties, while owners face obstacles in effectively showcasing their listings to the right audience. To address these issues, we developed a Property Recommendation System (PRS) — a unified digital platform that simplifies and automates property discovery and management.

PRS provides a user-friendly interface catering to different user roles:

- Admins (Property Owners): Can add, edit, or remove property listings and keep track of user activity.
- Registered Users (Buyers/Tenants): Can specify their requirements—such as preferred location, budget range, and property type—to receive smart, personalized recommendations, view detailed property information, and make bookings.
- Guests (Visitors): Can browse through available property listings without needing to register.

The system is developed using Java for backend processing and MySQL for secure and reliable data storage. Through this project, we highlight the practical application of object-oriented programming principles in creating an intelligent, efficient, and accessible solution for today's real estate challenges.

Existing System

Manual Property Handling

- At present, most property-related activities are still managed in a traditional way. Owners keep physical notes or rely on direct communication to record property details. Students and working individuals usually depend on brokers, friends, or word-of-mouth to find hostels, PGs, or rental rooms.
- Since there is no digital framework, information about availability, pricing, or occupancy often becomes outdated or inconsistent. Bookings and payments are managed informally, leaving no traceable proof in case of misunderstandings. This manual approach leads to delays, confusion, and a lack of transparency in property management.

Existing Online Solutions

- Although several online platforms exist for renting houses or booking hotels, they are mostly geared toward short vacations or family stays. Real estate websites list properties for sale or rent, but they do not cater to the shared-living or student accommodation segment.
- Most of these systems are generic and non-personalized, providing static listings instead of recommendations based on user needs. Therefore, there is still an absence of a specialized, intelligent system that simplifies property discovery and management—especially for students or budget-focused users.

Proposed System

Key Features of the Property Recommendation System

1. Intelligent Recommendation Engine:

Suggests properties based on user preferences such as location, price range, property type, and facilities using a smart matching algorithm.

2. Multi-User Access Control:

Admins can manage property listings and user activity; registered users can set preferences, view recommendations, and book properties; guests can browse listings without registration.

3. Personalized Search and Filtering:

Enables users to apply filters like location, rent, and amenities to refine search results and receive the most suitable recommendations.

4. Real-Time Property Updates:

Ensures that property status—available, booked, or sold—is automatically updated in the system to maintain accuracy.

Proposed System

5. Centralized Database Integration:

Employs MySQL to securely store property listings, user details, and recommendation data, ensuring consistency and reliability.

6. Object-Oriented Design:

Developed using OOP principles such as inheritance, polymorphism, and encapsulation to achieve modularity, security, and easy maintenance.

7. Transparency and Efficiency:

Reduces dependency on brokers, minimizes human intervention, and builds trust between owners and tenants through verified digital records.

Objective

Aim

The aim of the Property Recommendation System (PRS) is to develop an intelligent, automated platform that simplifies the process of searching, recommending, and managing properties. The system focuses on providing users with personalized property suggestions while ensuring secure data management and efficient interaction between owners and buyers/tenants.

Key Objectives

- Build a centralized platform for property recommendations and management.
- Implement a smart recommendation mechanism that matches user preferences like location, price, and property type.
- Provide role-based access for admins, registered users, and guests.
- Maintain real-time updates of property listings using MySQL for data integrity and accuracy.
- Ensure data security and validation across all operations.
- Apply OOP principles such as inheritance and encapsulation for modularity and scalability.
- Improve convenience, reliability, and transparency in property discovery and management.

Requirements

Software Requirements

- Java (JDK 17 or above) – For implementing backend logic using OOP principles.
- MySQL Database – To store property listings, user profiles, and recommendation data securely.
- JDBC Connector – To establish communication between Java and the MySQL database.
- IDE (Eclipse / IntelliJ IDEA / NetBeans) – For coding, compiling, and debugging the application

Hardware Requirements

- Minimum RAM: 4 GB or higher
- Processor: Intel Core i3 or above
- Storage Space: At least 500 MB free disk space

Requirements

Functional Requirements

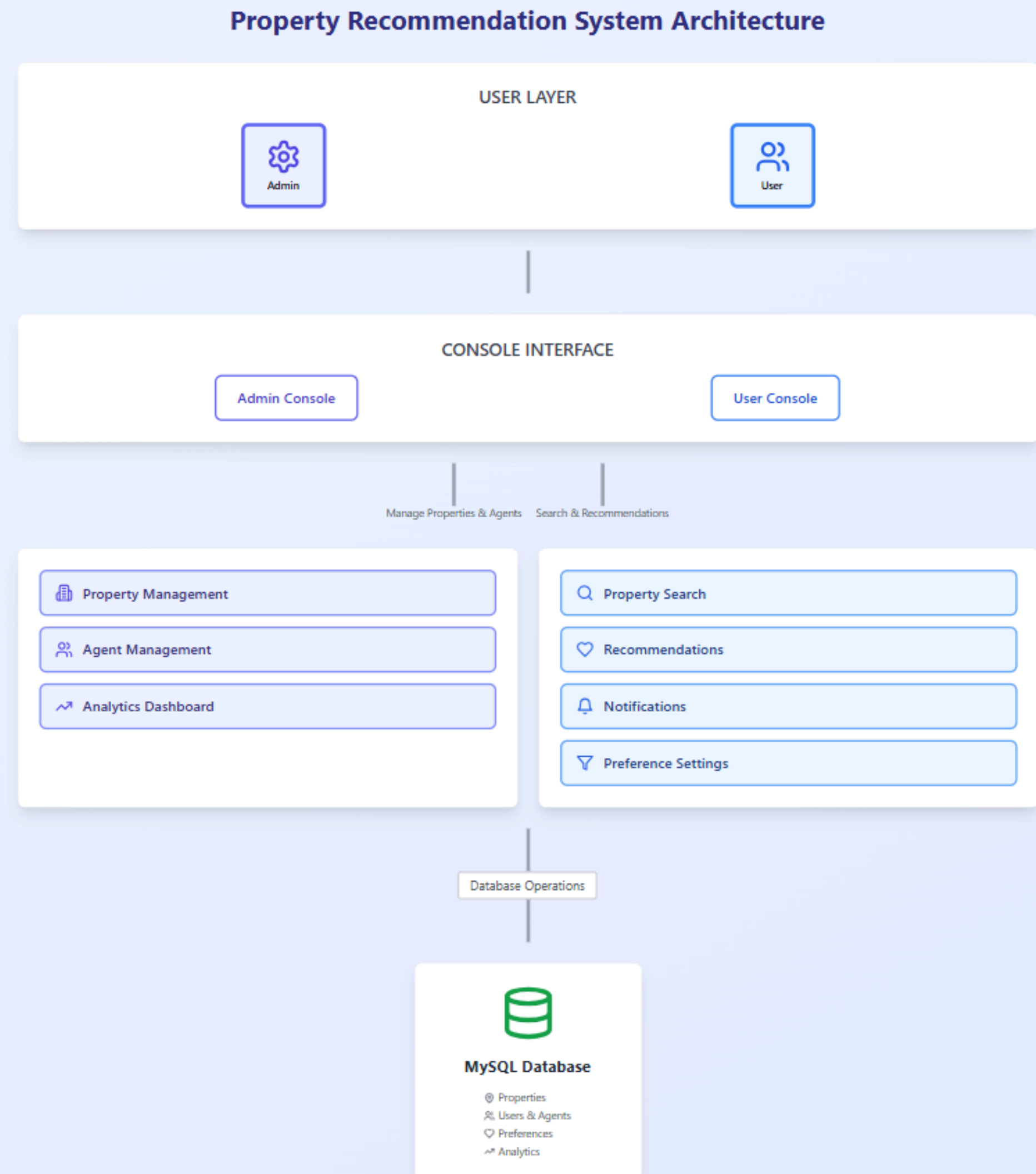
- User Registration & Login: Allows users to create accounts and securely access the system.
- Property Management: Admins can add, update, view, or delete property listings.
- Recommendation System: Generates property suggestions based on user preferences such as budget, location, and property type.
- Booking & Status Updates: Enables users to book properties, view availability, and cancel bookings when needed.
- Data Management: Ensures real-time synchronization of property and user information.

Non-Functional Requirements

- Reliability & Data Accuracy: Ensures stable performance and consistent data handling.
- Scalability: Supports future expansion of features and database size.
- Security: Implements authentication and validation to protect user data.
- User-Friendly Interface: Provides a simple, intuitive layout for easy navigation and usability.

Architecture Diagram

- Users (Admins and Buyers/Tenants) interact through dedicated console interfaces that connect to the Application Layer handling property management, search, and recommendation logic.
- All operations are supported by a centralized MySQL database, which securely stores and retrieves property, user, and booking data.
- This structure ensures modularity, scalability, and smooth communication between each layer.



Modules

- Admin Module
- User Module
- Guest Module
- Property Management Module
- Recommendation Module
- Booking and Payment Module

Module-Wise Description

- The **Admin Module** is responsible for managing all administrative operations within the system. It allows property owners or administrators to add, update, and delete property listings, monitor user activities, and oversee booking-related tasks.
- The **User Module** enables registered users to create profiles, enter preferences such as budget and location, receive personalized property recommendations, view details, and proceed with bookings or cancellations.
- The **Guest Module** provides access for unregistered visitors, allowing them to browse available properties and view essential information without needing to log in.

Module-Wise Description

- The **Property Management Module** focuses on handling CRUD operations (Create, Read, Update, Delete) for property data. It ensures that availability and pricing details are updated in real time, maintaining data accuracy across the platform.
- The **Recommendation Module** is the core of the system's intelligence. It filters property listings based on user-defined preferences and suggests the most relevant options using predefined recommendation logic.
- Finally, the **Booking and Payment Module** manages the process of property booking, cancellation, and payment tracking. It ensures secure transactions, data validation, and maintains complete booking records for transparency and accuracy.

Implementation

1)Entry point:

```
public static void main(String[] args) {
    while (true) {
        if (currentUser == null)
            showWelcomeMenu();
        else {
            switch (currentUser.getRole()) {
                case "ADMIN" -> showAdminMenu();
                case "SELLER" -> showSellerMenu();
                case "BUYER" -> showBuyerMenu();
                default -> System.out.println("Unknown role. Contact admin.");
            }
        }
    }
}
```

2)Welcome Menu:

```
//
private static void showWelcomeMenu() {
    System.out.println("\n=== Property Recommender ===");
    System.out.println("1. Login");
    System.out.println("2. Register");
    System.out.println("3. Continue as Guest");
    System.out.println("0. Exit");

    int c = InputUtil.readInt("Choose: ", -1);

    switch (c) {
        case 1 -> handleLogin();
        case 2 -> handleRegister();
        case 3 -> handleGuest();
        case 0 -> {
            System.out.println("Bye!");
            System.exit(0);
        }
        default -> System.out.println("Invalid choice.");
    }
}
```


3)JDBC Connection:

```
public class JDBCUtil {
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver"); // MySQL 8 driver
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBC driver not found", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DBConfig.URL, DBConfig.USER, DBConfig.PASS);
    }

    public static void closeQuietly(AutoCloseable c) {
        if (c != null) {
            try { c.close(); } catch (Exception ignored) {}
        }
    }
}
```

4)Recommendation Logic:

```
// Filter by locality and budget and then sort by price asc (simple ranking)
public List<Property> recommend(String locality, double maxPrice, int bhkPreference) {
    List<Property> candidates = propertyService.search(locality, maxPrice);
    // Score properties: match bhk gives extra score, lower price better rank.
    Map<Property, Integer> score = new HashMap<>();
    for (Property p : candidates) {
        int s = 0;
        if (p.getBhk() == bhkPreference) s += 10;
        // price-based score (lower price higher score)
        s += (int)(1000000 / (1 + p.getPrice()));
        score.put(p, s);
    }
    return score.entrySet().stream()
        .sorted(Map.Entry.<Property, Integer>comparingByValue().reversed())
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());
}
```


5)Booking Logic:

```
public Booking bookProperty(int propertyId, int userId, double amount) {
    Property p = propertyDAO.findById(propertyId);
    if (p == null || !p.isAvailable()) throw new RuntimeException("Property not available");
    Booking b = new Booking();
    b.setPropertyId(propertyId);
    b.setUserId(userId);
    b.setBookingDate(new Timestamp(System.currentTimeMillis()));
    b.setStatus("BOOKED");
    b.setAmount(amount);
    Booking saved = bookingDAO.save(b);
    // set property unavailable
    p.setAvailable(false);
    propertyDAO.save(p);
    return saved;
}

public boolean cancelBooking(int bookingId) {
    Booking b = bookingDAO.findById(bookingId);
    if (b == null || "CANCELLED".equals(b.getStatus())) return false;
    b.setStatus("CANCELLED");
    bookingDAO.save(b);
    // make property available
    Property p = propertyDAO.findById(b.getPropertyId());
    if (p != null) { p.setAvailable(true); propertyDAO.save(p); }
    return true;
}
```

6)Buyer Menu:

```
// -----
//  BUYER MENU
// -----
private static void showBuyerMenu() {
    System.out.println("\n=== Buyer Menu ===");
    System.out.println("1. Search / Recommend Properties");
    System.out.println("2. Book a Property");
    System.out.println("3. My Profile");
    System.out.println("4. Logout");

    int c = InputUtil.readInt("Choose: ", -1);

    switch (c) {
        case 1 -> searchAndRecommend();
        case 2 -> bookProperty();
        case 3 -> viewProfile();
        case 4 -> currentUser = null;
        default -> System.out.println("Invalid choice.");
    }
}
```


7) Seller Menu:

```
// -----  
// SELLER MENU  
// -----  
private static void showSellerMenu() {  
    System.out.println("\n=== Seller Menu ===");  
    System.out.println("1. Add Property");  
    System.out.println("2. Update My Property");  
    System.out.println("3. Delete My Property");  
    System.out.println("4. View My Properties");  
    System.out.println("5. Logout");  
  
    int c = InputUtil.readInt("Choose: ", -1);  
  
    switch (c) {  
        case 1 -> addPropertyForSeller();  
        case 2 -> updatePropertyForSeller();  
        case 3 -> deletePropertyForSeller();  
        case 4 -> viewMyProperties();  
        case 5 -> currentUser = null;  
        default -> System.out.println("Invalid choice.");  
    }  
}
```

8) Admin Menu:

```
// -----  
// ADMIN MENU  
// -----  
private static void showAdminMenu() {  
    System.out.println("\n=== Admin Dashboard ===");  
    System.out.println("1. Manage Properties");  
    System.out.println("2. Manage Users");  
    System.out.println("3. View All Bookings");  
    System.out.println("4. Logout");  
  
    int c = InputUtil.readInt("Choose: ", -1);  
  
    switch (c) {  
        case 1 -> manageProperties();  
        case 2 -> manageUsers();  
        case 3 -> viewBookings();  
        case 4 -> currentUser = null;  
        default -> System.out.println("Invalid choice.");  
    }  
}
```


Result

Buyer

```
=== Property Recommender ===    === Buyer Menu ===
1. Login                        1. Search / Recommend Properties
2. Register                    2. Book a Property
3. Continue as Guest          3. My Profile
0. Exit                        4. Logout
Choose: 1                     Choose: 2
Username: buyer1              Property ID to book: 1
Password: pass123             Enter payment amount: 20000000
Welcome Rahul Buyer           Booked successfully!
```


Result

Seller

```
=== Seller Menu ===
1. Add Property
2. Update My Property
3. Delete My Property
4. View My Properties
5. Logout
Choose: 4
Property[id=1, title=Metropolis, locality=Gachibowli, bhk=3, price=20000000.00, available=false]
```

Admin

```
=== Admin Dashboard ===
1. Manage Properties
2. Manage Users
3. View All Bookings
4. Logout
Choose: 3
Booking[id=1, propertyId=1, userId=4, status=BOOKED, amount=20000000.00]
```


Conclusion

The Property Recommendation System efficiently streamlines the process of discovering and booking residential properties, including rental apartments, PGs, and hostels. By leveraging Java-based object-oriented programming along with MySQL database management, the system ensures robustness, data integrity, and ease of maintenance. It bridges the gap between property seekers and owners by providing a user-friendly and transparent platform, overcoming the inefficiencies of traditional property search methods.

This project highlights the practical application of software engineering principles to develop a scalable, organized, and solution-oriented system. Looking ahead, the system can be further enhanced as a web or mobile application, incorporating features like location-based recommendations, personalized property suggestions, and secure online payment integration—transforming it into a complete digital ecosystem for property seekers.

The End

THANK YOU FOR LISTENING